

Week 2: Product and Category Management System

Scenario:

The e-commerce company needs to store and manage information about its products and categories. Each product belongs to a particular category, and the system should maintain product details such as product name, price, and stock availability.

1. Design Product and Category Tables

Category Table:

```
SQL> CREATE TABLE Category(Category_ID INT PRIMARY KEY,Category_Name VARCHAR(100) NOT NULL UNIQUE);
Table created.
```

Product Table:

```
SQL> CREATE TABLE Product(Product_ID INT PRIMARY KEY,Product_Name VARCHAR(100) NOT NULL,Price
DECIMAL(10,2) CHECK (Price > 0),Stock INT CHECK(Stock>= 0),Category_ID INT);
Table created.
```

The two tables for Category and Product have been created.

Now, we need to identify the relationship between the two tables and apply the required constraints.

2. Define Relationships and Constraints

Relationships:

One category can have many products.

The relationship between the Category and Product tables is **One-to-Many (1 → *)**.

After identifying the relationship, we need to understand which key is used to connect the two tables.

Before that, we need to understand some important **Key Constraints**.

PRIMARY KEY:

A Primary Key is a column or field that uniquely identifies each row in a table.

In these two tables, **Category_ID** can be used as the Primary Key in the Category table because each category has a unique ID. Similarly, **Product_ID** is the Primary Key in the Product table because it uniquely identifies each product.

FOREIGN KEY:

A Foreign Key is a key that is used to connect two tables.

Therefore, **Category_ID** in the Product table acts as the Foreign Key and connects it with the Category table.

We can also use other constraints such as **NOT NULL, CHECK, and UNIQUE**.

NOT NULL:

The NOT NULL constraint ensures that a column cannot have an empty or NULL value.

For example, the Product Name and Category Name should not be empty because every product and category must have a name.

CHECK:

The CHECK constraint is used to make sure that the values entered into a column satisfy a particular condition.

For example, the price of a product should always be greater than zero.

UNIQUE:

The UNIQUE constraint ensures that duplicate values are not entered into a column.

For example, if UNIQUE is applied to Category Name, two categories cannot have the same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary Key:

```
SQL> ALTER TABLE Category
2  ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);
Table altered.

SQL> ALTER TABLE Product
2  ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);
Table altered.
```

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);
Table altered.
```

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;
Table altered.
```

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_PRICE CHECK (Price > 0);  
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);  
Table altered.
```

3. Insert Values:

After creating the tables and applying the required constraints, we can insert sample records into the Product and Category tables.

Product Table:

PRODUCT_ID	PRODUCT_NAME	PRICE	STOCK	CATEGORY_ID
101	Laptop	55000.00	10	1
102	Mobile Phone	25000.00	20	1
103	T-Shirt	800.00	50	2
104	Jeans	1500.00	30	2
105	Novel	500.00	25	3
106	Mixer Grinder	3500.00	15	4

6 rows selected.

Like this, a few more records can be inserted into the table.

After Insertion Category Table:

```
SQL> INSERT INTO Product (Product_ID, Product_Name, Price, Stock, Category_ID) VALUES (107, 'Headphones', 2000, 12, 1);
1 row created.

SQL> SELECT * FROM Product;

PRODUCT_ID PRODUCT_NAME      PRICE STOCK CATEGORY_ID
-----
101 Laptop          55000.00  10      1
102 Mobile Phone    25000.00  20      1
103 T-Shirt          800.00    50      2
104 Jeans            1500.00   30      2
105 Novel            500.00    25      3
106 Mixer Grinder    3500.00   15      4
107 Headphones       2000.00   12      1

7 rows selected.
```

CRUD Operations:

Insert Record:

```
SQL> INSERT INTO Product (Product_ID, Product_Name, Price, Stock, Category_ID) VALUES (107, 'Headphones', 2000, 12, 1);
1 row created.

SQL> SELECT * FROM Product;

PRODUCT_ID PRODUCT_NAME      PRICE STOCK CATEGORY_ID
-----
101 Laptop          55000.00  10      1
102 Mobile Phone    25000.00  20      1
103 T-Shirt          800.00    50      2
104 Jeans            1500.00   30      2
105 Novel            500.00    25      3
106 Mixer Grinder    3500.00   15      4
107 Headphones       2000.00   12      1

7 rows selected.
```

Update Price:

```
SQL> UPDATE Product
  2 SET Price = 2200
  3 WHERE Product_ID = 107;

1 row updated.

SQL> SELECT * FROM Product
  2 WHERE Product_ID = 107;

PRODUCT_ID PRODUCT_NAME      PRICE STOCK CATEGORY_ID
-----
107 Headphones       2200.00  12      1
```

Delete a Product:

```
SQL> DELETE FROM Product WHERE Product_ID = 107;
```

```
1 row deleted.
```

```
SQL> SELECT * FROM Product;
```

PRODUCT_ID	PRODUCT_NAME	PRICE	STOCK	CATEGORY_ID
101	Laptop	55000.00	10	1
102	Mobile Phone	25000.00	20	1
103	T-Shirt	800.00	50	2
104	Jeans	1500.00	30	2
105	Novel	500.00	25	3
106	Mixer Grinder	3500.00	15	4

```
6 rows selected.
```

5. Generate Category-wise Product Reports

Write SQL queries to:

- Display all products under each category.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock FROM Category c JOIN Product p ON c.Category_ID = p.Category_ID;
```

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK
Electronics	Laptop	55000.00	10
Electronics	Mobile Phone	25000.00	20
Clothing	T-Shirt	800.00	50
Clothing	Jeans	1500.00	30
Books	Novel	500.00	25
Home Appliances	Mixer Grinder	3500.00	15

```
6 rows selected.
```

- Count the number of products in each category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Category c LEFT JOIN Product p ON c.Category_ID = p.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
Clothing	2
Home Appliances	1
Books	1
Electronics	2

- Find the highest-priced product in each category.

```
SQL> SELECT c.Category_Name, MAX(p.Price) AS Highest_Price FROM Category c JOIN Product p ON c.Category_ID = p.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	HIGHEST_PRICE
Clothing	1500
Home Appliances	3500
Books	500
Electronics	55000

- Display categories having more than 5 products.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Category c JOIN Product p ON c.Category_ID = p.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID) > 5;
```

```
no rows selected
```

- Find the average product price category-wise.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Average_Price FROM Category c JOIN Product p ON c.Category_ID = p.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVERAGE_PRICE
Clothing	1150
Home Appliances	3500
Books	500
Electronics	40000